

Test Plan

Requirement/Function	Partition	Inputs	Expected outputs	Description	Valid/invalid
FR1 — Generate parking spaces	Valid destination	Destination at (50.79, -1.09), maxResult=10	Result=10 ParkingSpace objects with ParkingService.generateForDestination returns spaces clustered around the destination.		
	Sort by distance	Result set, sortOrder=ParkingSortOrder.Distance	Result set ordered ascending by walking distance	filterParkingSpaces returns results sorted nearest first.	
	Sort by price	Result set, sortOrder=ParkingSortOrder.Price	Result set ordered ascending by price	Per-hour space appears first.	v
	Sort by safety score	Result set, sortOrder=ParkingSortOrder.SafetyScore	Result set ordered descending by safety score	Best space appears first; lighting and bay width contribute.	
	Empty destination key	Destination with empty name and lat/long list	Empty list, no exception	Robust to a destination that has not been searched yet.	
	All spaces filtered out	Spaces present, but every filter excluded	Empty list	filterParkingSpaces returns [] rather than throwing when no space matches	
FR2 — Refresh availability	Refresh reverses previous movement	Space with previousAvailableSpaces=4, refresh called	previousAvailableSpaces=2	refreshAvailability reverses the previous movement and stores the prior value.	
	Refresh increments a decreasing space	Space whose availability has been falling	refresh, availableSpaces is incremented	Demonstrates the alternating behaviour used for the demo.	
	Repeated refresh keeps changing value	Same space, refresh called twice	previousAvailableSpaces updated each time	Available value is always the value from the previous cycle.	
	Sold-out space stays at zero	Space with availableSpaces=0 and totalSpaces=10	refreshAvailability keeps availableSpaces=0	refreshAvailability keeps sold-out spaces at zero (no negative values).	
	Empty list input	refreshAvailability called with []	Returns [], no exception	Robust to refreshing when no spaces have been generated yet.	
FR3 — Filter parking results	Filter by price below max	maxPricePerHour=£3.00 with space prices £1.50-£5.00	Only spaces with price ≤ £3.00 returned	Price filter on the upper boundary.	v
	Filter by type	type=wide, mixed result set	Only spaces with type=wide returned	Type partition applied (standard/wide/disabled).	
	Filter by max walking distance	maxDistanceKm=0.5, mixed distances	Only spaces within 0.5 km returned	Distance partition on the upper boundary.	
	Filter requires lighting	requiresLighting=true, mixed hasLighting	Only spaces with hasLighting=true returned	Lighting filter added during Iteration 2.	
	All filters applied simultaneously	maxPrice + maxDistance + type + lighting	Only spaces satisfying every constraint returned	Verifies AND-composition of filters.	v
	Filter out full spaces	Space with availableSpaces=0	Excluded from results	Zero availability is implicitly invalid for the user.	
	Price above all spaces	maxPricePerHour=£0.01	Empty list returned	Lower-boundary invalid input.	i
	No matching type	type=disabled, no disabled spaces present	Empty list returned	Invalid partition where the requested type does not exist.	
FR4 — Parking details bottom sheet	Name renders	ParkingCard given a ParkingSpace with name 'Cathedral Car Park'	Card displays 'Cathedral Car Park'	ParkingCard widget test asserts the rendered name.	
	Walking distance and price line	ParkingResult with walkingDistance=0.34, pricePerHour=£2.00	Summary line reads '0.34 miles from 0.32 km · £2.00/hour'	Verifies the formatted summary line.	v
	Opening hours and safety score	ParkingSpace with openingHours='24/7', safetyScore=4	Card displays opening hours and safety score	Both fields are visible at a glance.	v
	Lighting yes / no	hasLighting=true vs false	Card shows 'Lighting Yes' or 'Lighting No'	Valid partitions of the lighting flag.	
	No availability badge	Space with availableSpaces=0	Badge reads 'No availability'	Invalid (unusable) state still rendered correctly.	
FR5 — Navigation to parking	Navigate button present	ParkingCard rendered for any space	Navigate button visible and tappable	Asserted in parking_card_test.dart.	v
	Navigate callback fires	User taps Navigate	onNavigate callback invoked with the space	Asserts the wiring between widget and URL launcher.	
	Manual: Google Maps opens	Tap Navigate on a real device	External Google Maps opens with the space	Tested manually in the demo (FR5 reduced to a deep link).	
	Manual: no network	Tap Navigate with no internet	OS shows its standard external-app graceful failure	Verified manually.	i
FR6 — Availability alerts	Detects new spaces gained	Two snapshots where one space was added	spacesWithNewAvailability returns the new space	Developed app alert card.	v

Requirement/Function	Partition	Inputs	Expected outputs	Description	Valid/invalid
	No improvement	Snapshots identical for every space	Empty list (no alert)	Returns empty when no spaces improved.	v
	Equal availability counts as no change	Snapshots with same available spaces	Empty list	Equal availability does not raise an alert.	v
	Decrease ignored	Snapshot where a space went 5 → 4	Empty list	Alerts only fire on improvements, never on decreases.	v
	Empty snapshots	Either side empty	Empty list, no exception	Robust to first-run state where there is no previous snapshot.	v
FR7 — Walking distance (Haversine)	Identical coordinates	(50.79, -1.09) to (50.79, -1.09)	0.00 km	Identical coordinate pair returns zero.	v
	Short distance ~1.2 km	Two points ~1.2 km apart	Approximately 1.2 km	Verifies the formula on a short city-scale distance.	v
	Medium distance ~10 km	Two points ~10 km apart	Approximately 10 km	Verifies the formula on a medium distance.	v
	Cross-country distance	London to Edinburgh coordinates	Large positive value (~530 km)	Verifies the formula on a long distance.	v
	Symmetric: A→B = B→A	Any two distinct points	Both directions return the same distance	Asserts mathematical symmetry of the Haversine formula.	v
	Negative latitude and longitude	Southern/western hemisphere coordinates	Positive distance returned	Handles negative inputs correctly.	v
	Never returns negative	Any inputs	Result ≥ 0	Distance must always be non-negative; invalid by definition otherwise.	v
Authentication — sign up	Valid sign-up	name='Fardean', email='f@p.com', AppUser = Passed with matching fields	Valid sign-up produces a correctly populated AppUser.		v
	Auth stream emits new user	Same as above	authStateChanges emits the new AppUser	AppUser reactive auth-state propagation.	v
	Multiple independent sign-ups	Two different valid users	Both users are stored independently	No cross-talk between accounts.	v
	Email is normalised	email='F@P.COM'	Stored as 'f@p.com'	Email stored lower-case so sign-in is case-insensitive.	v
	Duplicate email	Sign up twice with the same email	Throws AuthException on the second attempt	Duplicate email partition is invalid.	i
	Invalid email format	email='notAnEmail'	Sign-up rejected at the screen level	AuthScreen widget test asserts the validation error.	v
	Password too short	password='abc'	Sign-up rejected at the screen level	Boundary on minimum password length.	v
	Empty name	name=''	Sign-up rejected at the screen level	Boundary on required field.	i
Authentication — sign in / sign out	Valid sign-in	email and password matching a registered user	AppUser returned and auth stream emits user	Valid credentials path.	v
	Case-insensitive sign-in	email='F@P.COM' after sign-up with 'f@p.com'	Sign-in succeeds	Confirms email normalisation extends to sign-in.	v
	Sign out	Signed-in user calls signOut()	authStateChanges emits null	Sign-out flow returns to AuthScreen via AuthGate.	v
	Sign in after sign out	Same credentials reused	Sign-in succeeds and user is restored	Round-trip through the auth state machine.	v
	Unknown email	email='notRegistered@x.com'	Throws AuthException	Invalid credentials partition.	i
	Wrong password	Correct email, wrong password	Throws AuthException	Invalid credentials partition.	i
	No stored session at startup	Fresh app, no prior sign-in	authStateChanges emits null	First-run partition is treated as signed-out.	v
Geocoding — destination search	Valid response	Nominatim returns a list with one result	Destination object with correct name and lat/long	Standard happy path.	v
	First result wins	Nominatim returns multiple results	Returns only the first result	Deterministic disambiguation rule.	v
	Lat/Ing parsed as doubles	Coordinates come back as strings	Parsed to double correctly	Type coercion verified.	v

Requirement/Function	Partition	Inputs	Expected outputs	Description	Valid/invalid
	Southern/western hemisphere	Negative lat/lng in response	Destination preserves signed values	Negative coordinates handled.	v
	Empty result array	Nominatim returns []	Throws Exception	Invalid search term partition.	i
	HTTP 404	Network returns 404	Throws Exception	Server-error partition.	i
	HTTP 500	Network returns 500	Throws Exception	Server-error partition.	i
	Unknown extra fields	Response includes fields the parser doesn't know	Destination returned, extras ignored	Forward-compatible parsing.	v
Repository — persistence	Empty fetch	Empty repository, fetchParkingSpaces(key)	Res([])	MemoryParkingRepository and FirestoreParkingRepository both pass.	
	Store and retrieve one space	replaceParkingSpaces with [space], Return [space]	Res([space]) with all fields preserved	Single-item round-trip.	v
	Store and retrieve many spaces	replaceParkingSpaces with 10 spaces, Return all	Res([spaces]) in any order	Multi-item round-trip.	v
	Replace overwrites previous	Store [A], then replace with [B]	fetch returns only [B]	Replace-all semantics.	v
	Replace with empty clears	Store [A,B], then replace with []	fetch returns []	Allows the UI to clear the dataset.	v
	Independent destination keys	Two different destinationKey values	Each key's data is isolated	Keys partition the dataset cleanly.	v
	Unknown key fetch	fetchParkingSpaces('never-stored')	Returns []	Robust to keys the repo has never seen.	
	Returned list is a copy	Mutating the returned list	Repository's internal state is unaffected	Defensive copy partition.	i
Models — parsing and serialisation	ParkingSpace.fromMap complete	Map with every field present	Object with matching values	Standard parse path.	v
	ParkingSpace.fromMap missing fields	Map missing optional fields	Object uses default values	Default-value partition.	v
	ParkingSpace.copyWith partial update	copyWith(availableSpaces: 7)	Only availableSpaces changes; original unchanged	Immutability partition.	v
	ParkingSpace.toMap excludes id	Object with id='abc'	Resulting map does not contain 'id'	id is the Firestore doc key, never a field.	
	Destination.key strips special characters	name='Portsmouth, UK!'	key='portsmouth-uk'	Slug rules for Firestore keys.	v
	Destination.key falls back to coords	name="" (empty)	key derived from lat/lng	Empty-name partition.	i
	AppUser accepts empty strings	uid="", email="", name=""	Object constructed without throwing	Permissive constructor partition.	i
Widget — AuthScreen	Default state shows Log in heading	AuthScreen launched	Heading reads 'Log in'	Initial mode is sign-in.	v
	Switch to sign-up reveals fields	Tap 'Sign up' toggle	Full-name and Confirm-password fields appear	Mode-toggle partition.	v
	Switch back to log-in hides fields	From sign-up, tap 'Log in' toggle	Full-name field removed	Reverse-toggle partition.	v
	Empty email submit	Submit form with email field blank	Inline error rendered	Validation error on missing input.	i
	Invalid email format	email='notAnEmail'	Inline error rendered	Validation error on malformed input.	i
	Short password	password='abc'	Inline error rendered	Boundary on password length.	i
	Passwords do not match	password!=confirmPassword in sign-up	Inline error rendered	Cross-field validation.	i
Widget — ParkingHomeScreen and AuthScreen	Title bar shows user name in app bar	Home screen with signed-in user	App bar shows app title and user name	Basic chrome rendering.	v
	Search field and preset chips visible	Home screen rendered	Search field, reset filters, preset chips all visible	Initial controls render.	v

Requirement/Function	Partition	Inputs	Expected outputs	Description	Valid/invalid
	Results count after loading	After parking generated	Results count text visible	Loading-state transitions to results state.	
	Refresh button does not crash	Tap Refresh availability	No exception, UI updates	Smoke test for the refresh wiring.	v
	AuthGate routes signed-in user	Auth stream emits AppUser	ParkingHomeScreen rendered	Navigation guard partition.	v
	AuthGate routes signed-out user	Auth stream emits null	AuthScreen rendered	Negative routing partition.	v
	Empty search submits snackbar	Submit search with blank field	Snackbar telling user to enter a destination	Invalid input partition.	i
	Auth stream pending shows AuthScreen	Before auth state resolves	AuthScreen shown (rather than crash)	Loading-state partition.	i

Summary

Smart Parking Finder — Test Plan Summary	
Total blocks in plan	14
Total partition rows	93
Automated test functions in the suite	162
Automated test files	15
Run command	flutter test --coverage
CI workflow	https://github.com/fardean08/Group6DAPPCW/actions
Last full run	17 May 2026 — 162 passed, 0 failed (15s)